



AI-2002: Artificial Intelligence

Final Project Report

Submitted by:

Ayman Irshad 23f-0724

Rida Zubair 23i-2590

Maimoona Hanan 23i-2544

Submission Date: 10th May, 2026

Table of Contents

.....	1
Executive Summary	3
1. Introduction	3
1.1 Problem Statement.....	4
1.2 Scope of the Project.....	4
2. Objectives and Requirement Mapping	4
3. System Architecture.....	4
4. Shared Grid Model	5
5. CSP-Based Layout Validation	5
6. Fleet Planning	6
7. Delivery Path Planning.....	6
8. Real-Time Disruption Handling	7
9. Required 20-Step Simulation Scenario	8
10. Demand Forecasting	9
11. Anomaly Detection	11
12. Dashboard Interface	12
13. Validation Checks and Output Artifacts	12
14. Conclusion.....	13

AeroNet Lite

Autonomous Drone Delivery Simulation with CSP, Fleet Planning, A Routing, Real-Time Replanning, Demand Forecasting, and Anomaly Detection*

Executive Summary

AeroNet Pro is an implementation of the AeroNet Lite autonomous drone delivery simulator. The system models a small smart-city environment as a 10 x 10 grid and integrates multiple artificial-intelligence techniques into a single working dashboard. The project validates the grid layout using constraint-satisfaction rules, selects a drone fleet under a budget, plans safe drone routes using A* search and related algorithms, handles real-time no-fly disruptions through replanning, forecasts demand using regression, and detects drone flight anomalies using classification.

The final implementation demonstrates a complete 20-step simulation scenario. The simulation initializes the city grid, verifies layout safety, selects 3 light drones and 1 heavy drone, plans delivery routes, activates a no-fly disruption, reroutes affected drones, runs demand forecasting, detects telemetry anomalies, and produces a final operational summary. The observed dashboard results show CSP PASS, 4 selected drones, budget usage of 4800, total demand of 246.75, and a 20-step simulation outcome of 4 completed deliveries, 3 rerouted paths, 0 delayed deliveries, 0 failed deliveries, and 2 queued deliveries.

The machine-learning results are also reported through the dashboard and exported output files. Demand forecasting achieved MAE = 44.021 and RMSE = 65.805 using a Random Forest Regressor, while anomaly detection selected a Decision Tree classifier with accuracy = 0.986. These results show that the simulator satisfies the required AI coverage: CSP, heuristic optimization, search, real-time replanning, regression, and classification.

1. Introduction

Autonomous drone delivery systems require route planning, safety checking, demand awareness, and fault detection before they can operate reliably in urban environments. A full real-world drone delivery system is too complex for a semester project; therefore, this work implements a simplified but complete simulation. The city is represented as a 10 x 10 grid in which every cell stores its zone type, population density, delivery demand, hub availability, charging availability, medical pickup status, and no-fly status.

The objective is not to deploy a real drone system but to demonstrate how several AI concepts can be combined in one integrated application. The simulator uses CSP-based validation for layout safety, heuristic fleet planning, A* search for routing, dynamic replanning for disruptions, regression for demand forecasting, and classification for anomaly detection. A Streamlit dashboard provides the visual interface for testing the system.

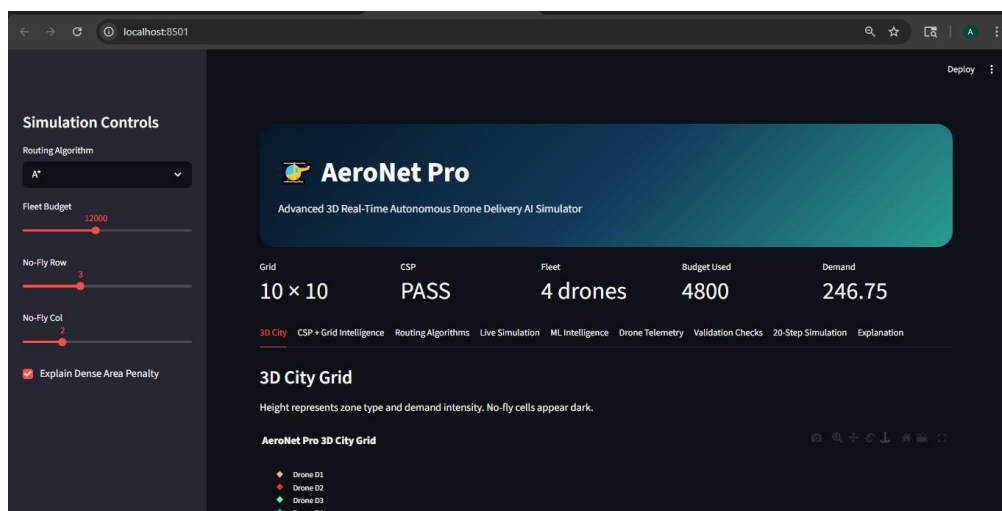


Figure 1. Dashboard overview showing grid size, CSP status, selected fleet, budget usage, and demand summary.

1.1 Problem Statement

The problem addressed in this project is how to simulate an intelligent drone delivery system that can operate safely and adaptively inside a simplified urban grid. The system must verify that the city layout is operationally valid, select a suitable drone fleet under a budget, generate safe delivery routes, respond to no-fly disruptions, estimate delivery demand, and detect abnormal telemetry behavior.

1.2 Scope of the Project

The scope is intentionally limited to a small grid-based environment. The final implementation uses a 10 x 10 grid, two drone types, a fixed set of delivery requests, simple no-fly disruptions, explainable ML models, and a dashboard for visualization. This makes the system feasible to implement and defend while still covering the required AI concepts.

2. Objectives and Requirement Mapping

Objective	Implementation in AeroNet Pro
Validate city layout	CSP rules R1-R4 check industrial safety, residential hub coverage, hub charging access, and hospital medical pickup access.
Select fleet under budget	Heuristic fleet selector chooses light and heavy drones based on total demand and budget.
Plan safe routes	A*, Dijkstra, and Weighted A* are implemented for route planning through the grid.
Avoid no-fly cells	Routing skips all cells where no_fly = True.
Handle disruptions	The simulation activates a new no-fly cell and reroutes affected drones.
Forecast demand	Random Forest Regressor predicts demand using Bike Sharing Demand features.
Detect anomalies	Decision Tree and Random Forest classifiers detect telemetry anomaly classes.
Provide visual demo	Streamlit dashboard contains city, CSP, routing, ML, telemetry, validation, and 20-step simulation tabs.

3. System Architecture

The simulator is organized into independent modules that communicate through a shared grid model. The shared model is the central data structure; all modules read from it, and selected modules update it during simulation. This design keeps the implementation simple and easy to explain during demonstration.

Module	File	Main Responsibility
Grid model	grid_model.py	Creates the 10 x 10 grid, cell attributes, hubs, charging pads, medical pickup points, no-fly cells, and demand values.
CSP validator	csp_validator.py	Checks layout rules R1-R4 and returns pass/fail messages.
Fleet selector	fleet.py	Selects light and heavy drones under budget and builds drone objects.
Route planner	routing.py	Plans safe routes using A*, Dijkstra, and Weighted A*.
Simulation	simulation.py	Generates deliveries, applies disruption, reroutes drones, and runs the integrated 20-step scenario.
ML pipeline	ml_models.py	Runs demand forecasting and anomaly classification.
Visualization	visual3d.py	Builds 3D city and route visualizations.
Dashboard	app.py	Provides the Streamlit interface.

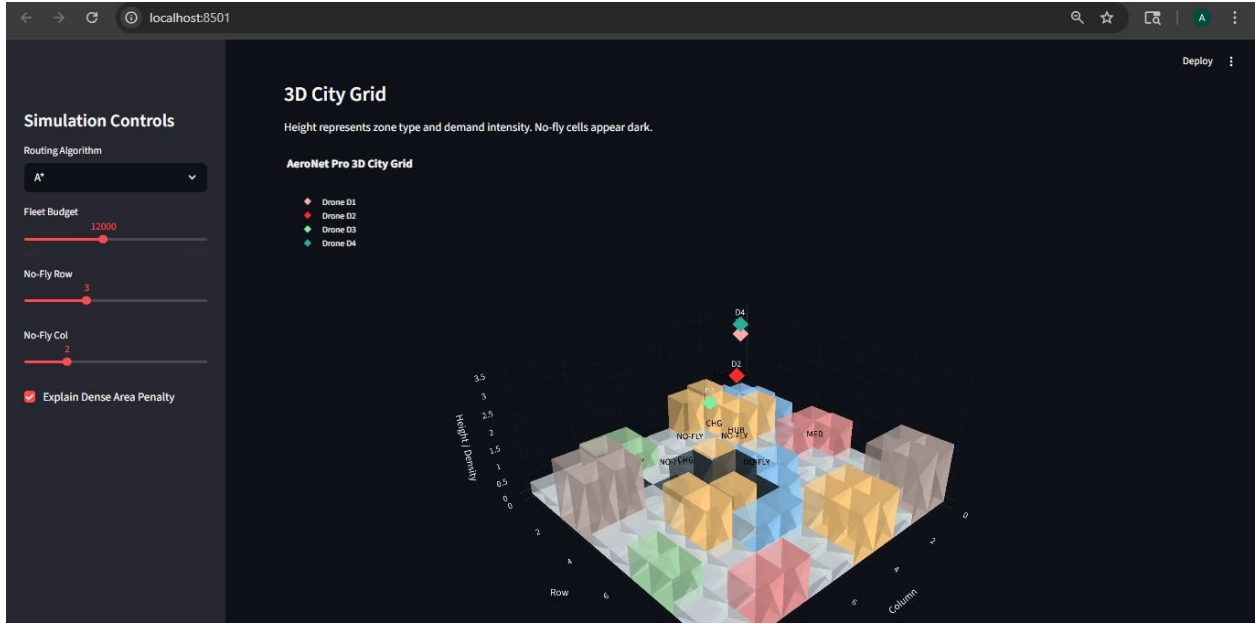


Figure 2. 3D city grid visualization. Height represents zone type and demand intensity, while dark cells indicate no-fly regions.

4. Shared Grid Model

The shared grid model is implemented as a 10 x 10 list of Cell objects. Each cell stores its row, column, zone, density, density level, estimated demand, hub flag, charging flag, medical pickup flag, and no-fly flag. Population-density values are loaded from the provided city population density dataset when available; otherwise, the implementation can fall back to generated density values. Demand is computed using a zone-weighted density formula so that commercial and residential areas can produce stronger delivery demand than open fields or industrial cells.

The manually controlled zone layout keeps the CSP rules explainable and reproducible. The grid includes hubs at (1,1), (5,6), and (4,4), charging pads at (1,2), (5,7), and (4,5), a medical pickup near the hospital at (5,1), and initial no-fly cells at (3,6), (4,6), (5,5), (6,5), and (2,7).

	cell	zone	density	density_level	demand	hub	charging	no_fly
0	0 0	Residential	1123	Low	1.12	☐	☐	☐
1	0 1	Residential	1394	High	13.94	☐	☐	☐
2	0 2	Commercial	12256	High	15.32	☐	☐	☐
3	0 3	Commercial	4530	High	5.66	☐	☐	☐
4	0 4	Open Field	4663	High	0.7	☐	☐	☐
5	0 5	Open Field	1818	Low	0.27	☐	☐	☐
6	0 6	School	4965	High	3.72	☐	☐	☐
7	0 7	School	4660	High	3.5	☐	☐	☐
8	0 8	Open Field	3489	Medium	0.52	☐	☐	☐
9	0 9	Open Field	3452	Medium	0.52	☐	☐	☐

Figure 3. Grid intelligence table showing cell-level zone, density, demand, hub, charging, and no-fly attributes.

5. CSP-Based Layout Validation

The layout validator behaves as a CSP-style constraint checker. Instead of automatically generating the complete city, the system verifies whether the manually defined grid satisfies the required constraints. Each rule is checked

independently, and all violations are collected instead of stopping at the first error. This makes the output more useful for debugging and reporting.

Rule	Constraint	Implementation
R1	Industrial cells must not be directly adjacent to schools or hospitals.	For every industrial cell, four-neighbor adjacency is checked.
R2	Every residential cell must be within 3 Manhattan cells of a drone hub.	Distance from every residential cell to all hubs is computed.
R3	Every drone hub must have a charging pad within 2 cells.	Distance from each hub to each charging pad is computed.
R4	At least one hospital must have a medical pickup point within 1 cell.	Hospital cells are checked against medical pickup cells.

The dashboard result shows that all four CSP rules passed. This confirms that the grid layout is valid before route planning and fleet operations begin.

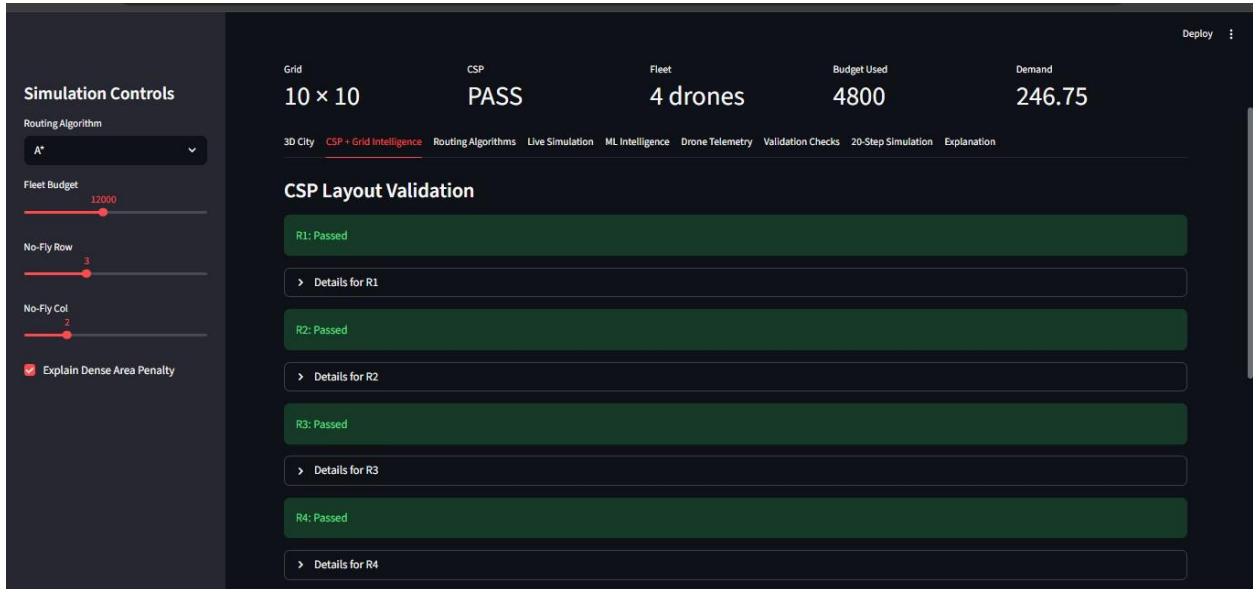


Figure 4. CSP layout validation results showing R1-R4 passed.

6. Fleet Planning

Fleet planning is handled as a simple optimization-style decision. The system supports two drone categories: light drones and heavy drones. Light drones are cheaper and suitable for small packages, while heavy drones are more expensive and suitable for heavier or medical deliveries. The implemented selector uses total grid demand and the available budget to choose the fleet composition. In the observed dashboard run, the selected fleet consisted of 3 light drones and 1 heavy drone, with total cost 4800 under the configured budget of 12000.

Drone Type	Cost	Payload	Range/Role in Project
Light Drone	1000	2 kg	Small parcels and nearby deliveries
Heavy Drone	1800	5 kg	Medical or heavier delivery support

The final dashboard summary reports 4 total drones and budget usage of 4800. This satisfies the requirement of selecting a small drone fleet under a fixed budget.

7. Delivery Path Planning

Delivery routing is one of the central AI components of the project. Each delivery is represented as a complete route: hub to pickup, pickup to drop-off, and drop-off back to hub. Each segment is solved using the selected routing algorithm. The implementation supports A*, Dijkstra, and Weighted A*, allowing the dashboard to compare algorithm behavior on the same routing task.

The routing state is a grid coordinate, and actions are limited to four-direction movement: up, down, left, and right. No-fly cells are treated as blocked nodes. A* uses Manhattan distance as the heuristic because the drone moves on a four-neighbor grid. The route planner also includes a dense-area penalty so that high-density and medium-density zones can be treated as more expensive to cross when the option is enabled.

Algorithm	Status	Cost	Route Length	Expanded Nodes
A*	Success	41.4	33	46
Dijkstra	Success	41.4	33	68
Weighted A*	Success	41.4	33	37

The comparison shows that all three algorithms found a route with the same cost and length. However, Weighted A* expanded the fewest nodes, while Dijkstra expanded the most. This is expected because Dijkstra performs uninformed search, whereas A* and Weighted A* use heuristic guidance.

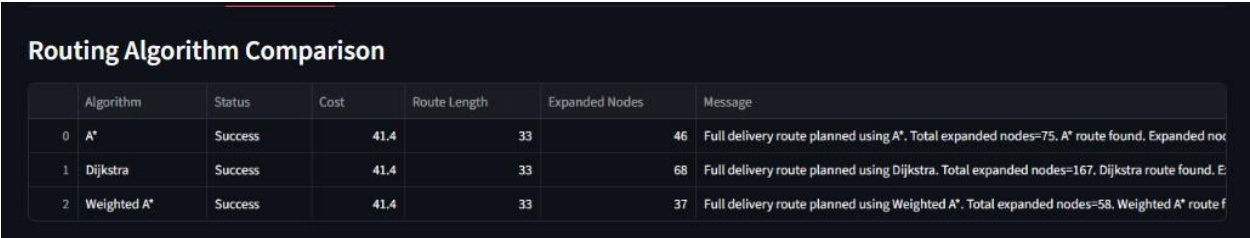


Figure 5. Routing algorithm comparison between A*, Dijkstra, and Weighted A*.

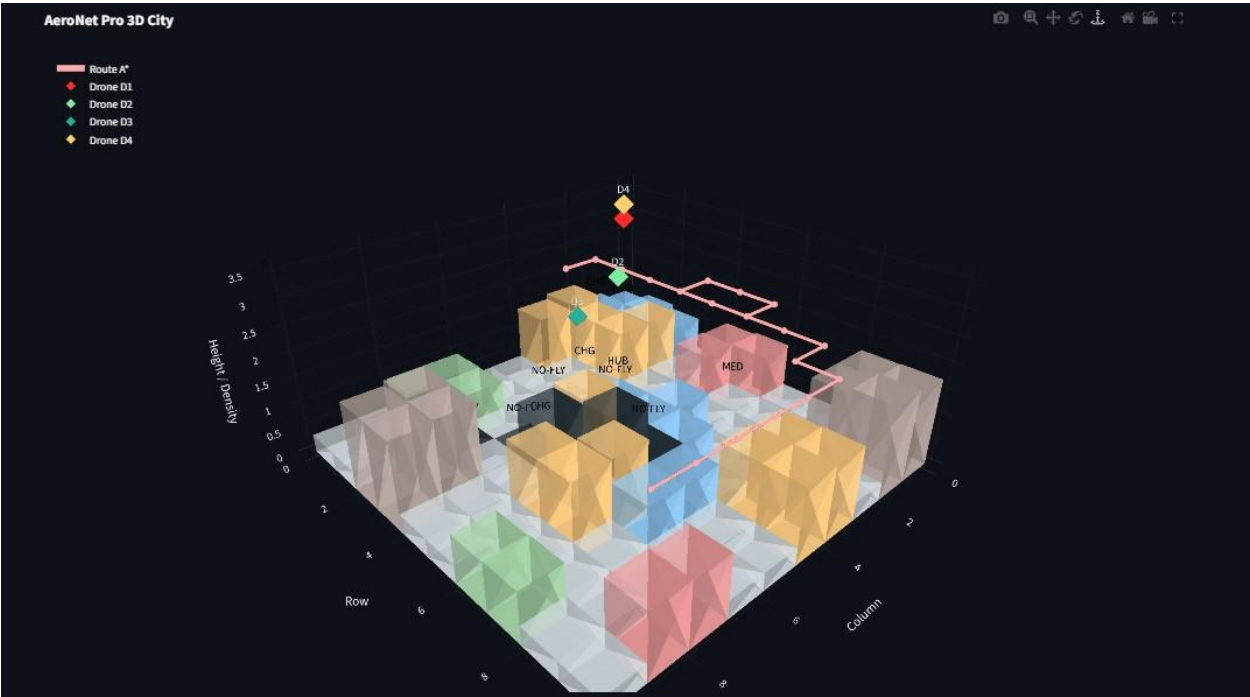


Figure 6. A* route visualization over the 3D city grid.

8. Real-Time Disruption Handling

The disruption handler simulates a realistic operational issue: a cell can become a no-fly zone during the mission. When this occurs, the simulator marks the selected grid cell as blocked and checks all current drone routes. If any route crosses the disrupted cell, the route is recomputed from the drone location toward the next goal using the selected routing algorithm.

The final demonstration used no-fly cell (3,2), which produced visible rerouting. In the live simulation event log, drones D1, D2, and D3 were rerouted successfully after the no-fly disruption. D1 was then marked as returning to hub after a telemetry safety check.

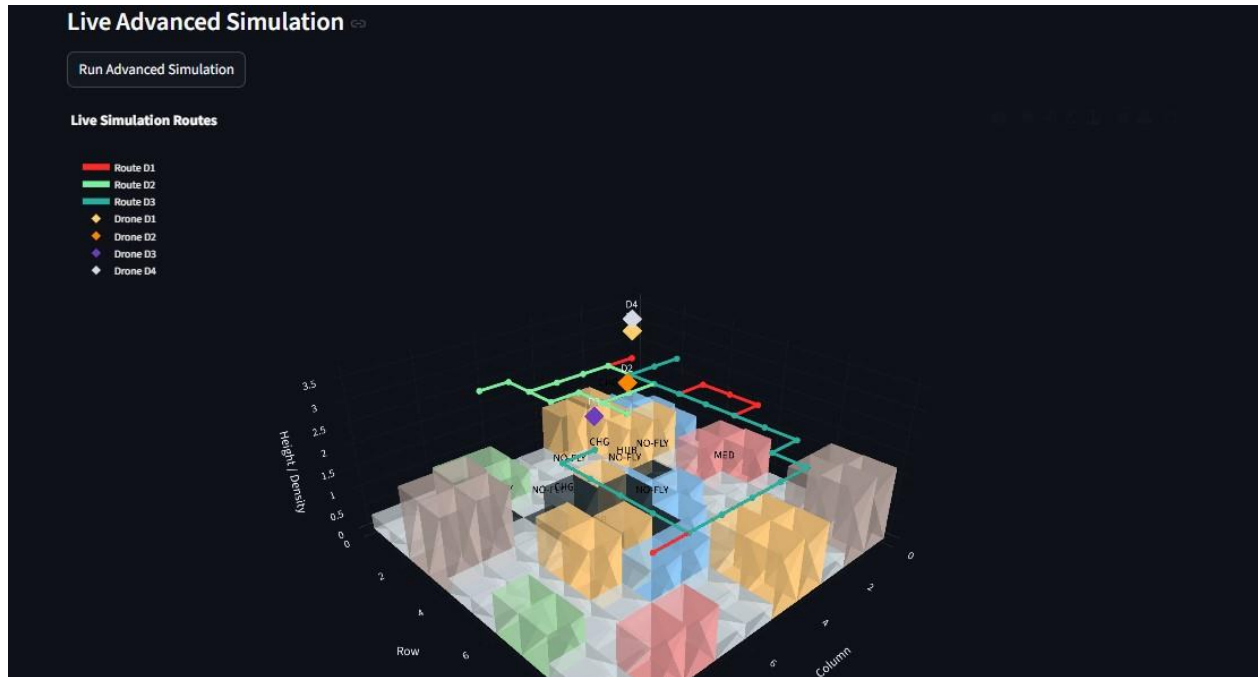


Figure 7. Live simulation event log and drone status after disruption and rerouting.

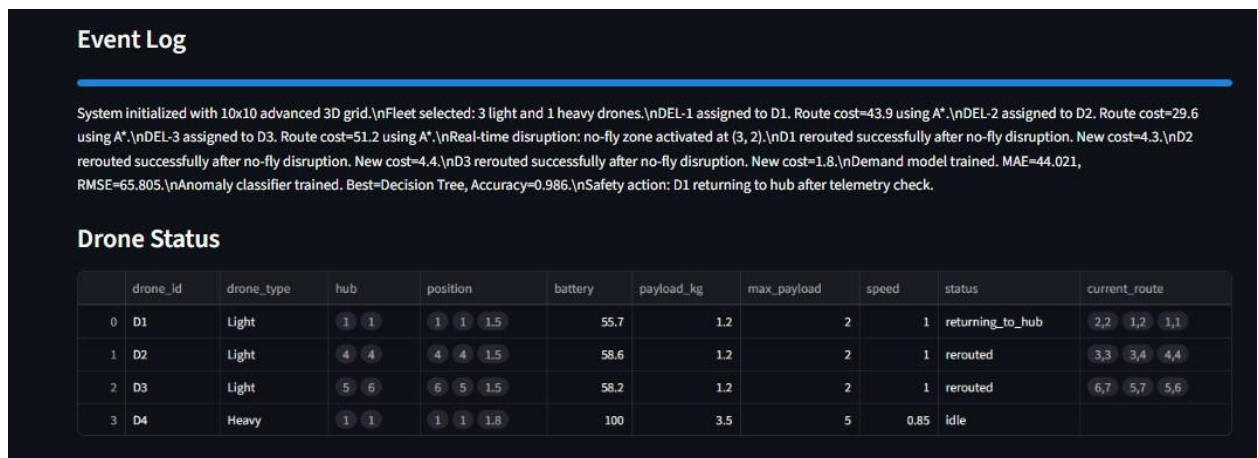


Figure 8. Drone status table after simulation showing rerouted and returning-to-hub states.

9. Required 20-Step Simulation Scenario

The project guide requires a 20-step simulation scenario. A separate dashboard tab was added to satisfy this requirement clearly. The 20-step scenario integrates the full pipeline: grid initialization, CSP validation, fleet planning, delivery generation, route planning, drone movement, no-fly disruption, rerouting, demand forecasting, anomaly detection, safety response, and final summary.

Step Range	Simulation Activity
1-3	Initialize grid, validate layout using CSP, and select fleet under budget.
4-6	Generate delivery requests, assign deliveries, and compute routes.
7-10	Move drones along planned paths.
11	Activate a no-fly disruption at the selected cell.
12-14	Check affected routes and reroute drones using A*.
15-17	Run demand forecasting and review demand/fleet status.
18	Run anomaly detection on drone telemetry.
19	Apply safety response by forcing a drone to return to hub.
20	Print final summary of completed, rerouted, delayed, failed, and queued deliveries.

The final 20-step summary produced 4 completed deliveries, 3 rerouted deliveries, 0 delayed deliveries, 0 failed deliveries, and 2 queued deliveries. This demonstrates that the integrated system can continue operation after a disruption without mission failure.

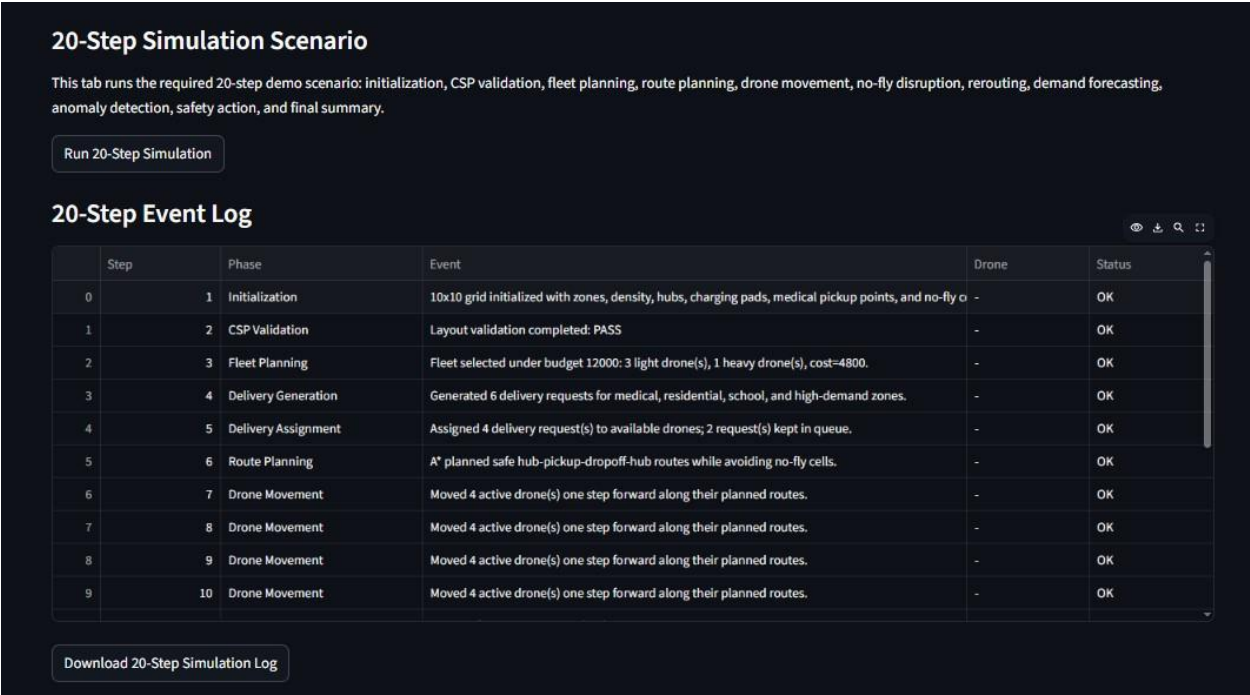


Figure 9. 20-step simulation event log showing initialization, CSP validation, fleet planning, routing, movement, and other phases.

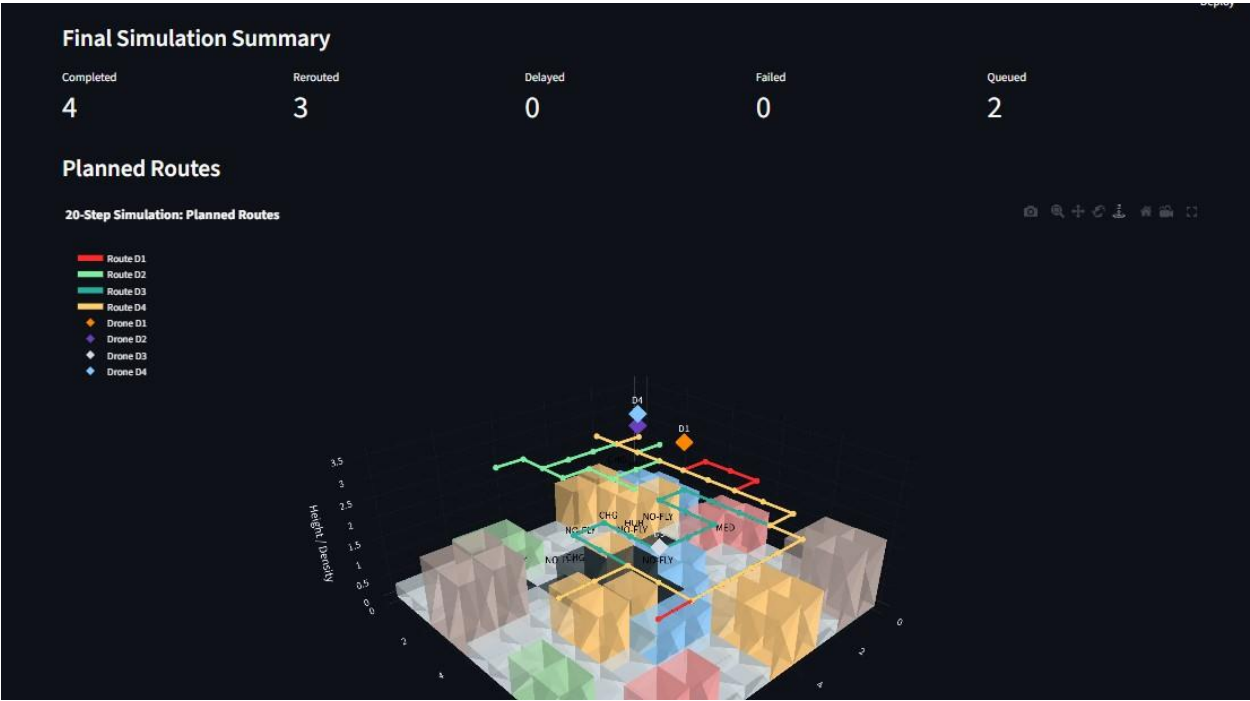


Figure 10. Final 20-step simulation summary and planned route visualization.

10. Demand Forecasting

Demand forecasting provides the regression component of the project. The implementation uses the Bike Sharing Demand dataset as a proxy for delivery demand because it contains time-based demand behavior and is easy to process within the semester-project scope. The selected features include season, holiday, working day, weather,

temperature, humidity, windspeed, hour, day of week, and month. A Random Forest Regressor is trained and evaluated using MAE and RMSE.

Metric	Value
Model	Random Forest Regressor
MAE	44.021
RMSE	65.805
Mean Predicted Demand	192.179
Most Important Feature	hour

The feature-importance result shows that hour is the strongest predictor, followed by temperature, humidity, working day, month, and day of week. This is reasonable because demand in time-based mobility or delivery datasets usually changes strongly across hours of the day.

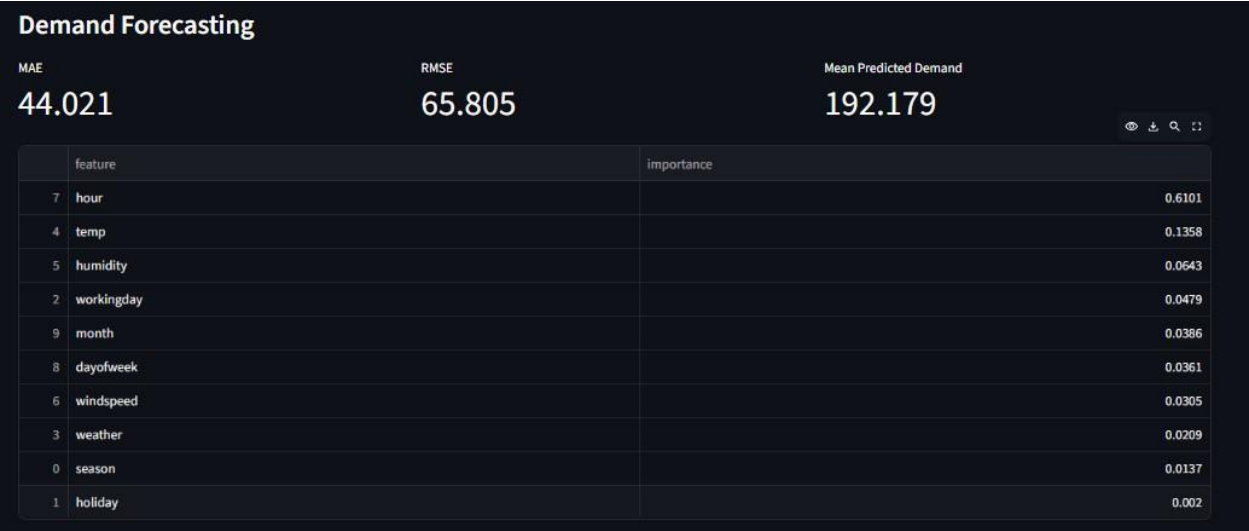


Figure 11. Demand forecasting metrics and feature-importance table.

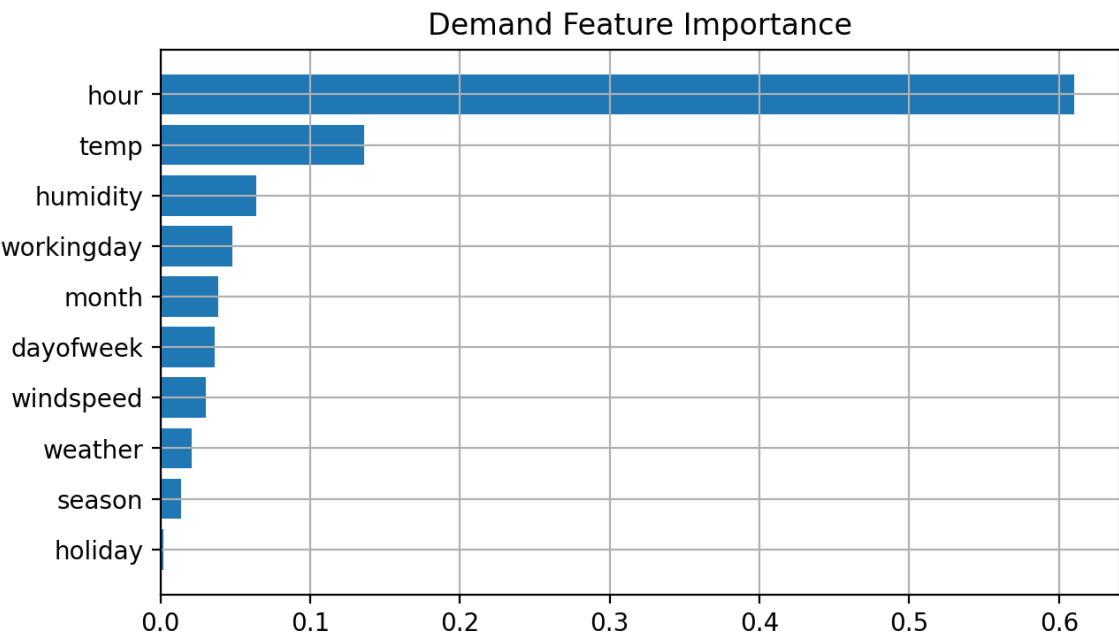


Figure 12. Demand feature importance plot showing hour as the dominant predictor.

11. Anomaly Detection

Anomaly detection provides the classification component of the project. The implementation uses drone operation log features such as battery remaining, GPS accuracy, wind speed, altitude, flight duration, and distance flown. Rule-based anomaly labels are generated from telemetry behavior. For example, low battery can indicate a battery anomaly, poor GPS accuracy or obstacle encounters can indicate route anomaly, and abnormal wind or altitude can indicate sensor/weather anomaly.

Metric	Value
Best Model	Decision Tree
Accuracy	0.986
Classes shown in dashboard	normal, route_anomaly, sensor_weather_anomaly
Model comparison	Decision Tree and Random Forest both reached 0.986 accuracy

The confusion matrix shows strong class-level performance. The model correctly classified 28 normal samples, 59 route-anomaly samples, and 49 sensor/weather anomaly samples. Only two visible off-diagonal errors appear in the displayed matrix, which explains the high accuracy.

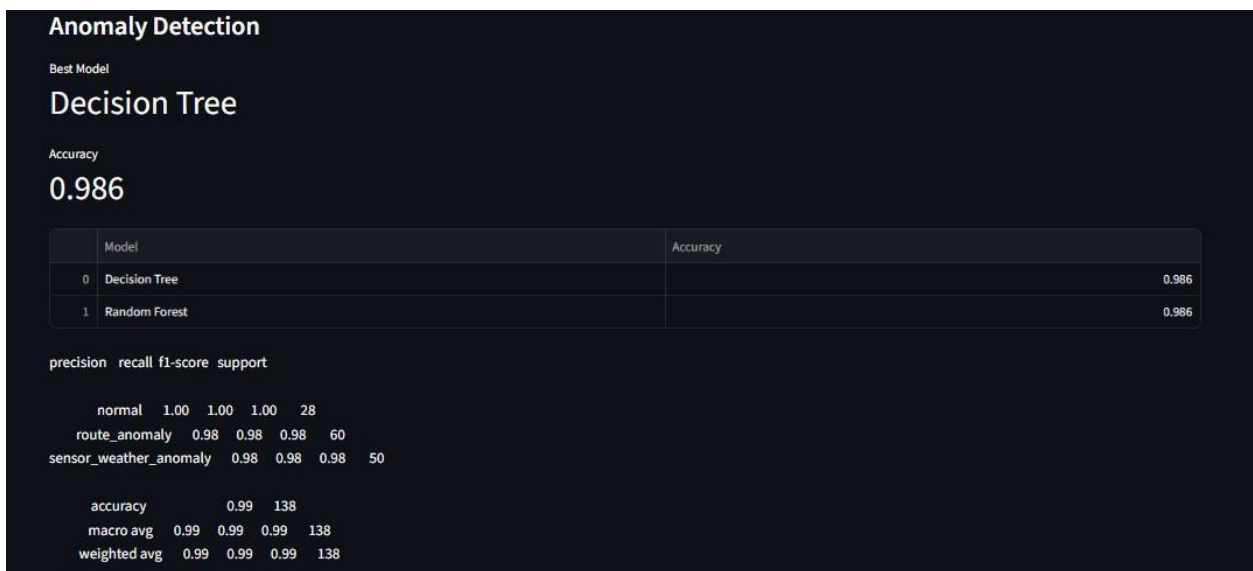


Figure 13. Anomaly detection metrics, classification report, and model comparison.

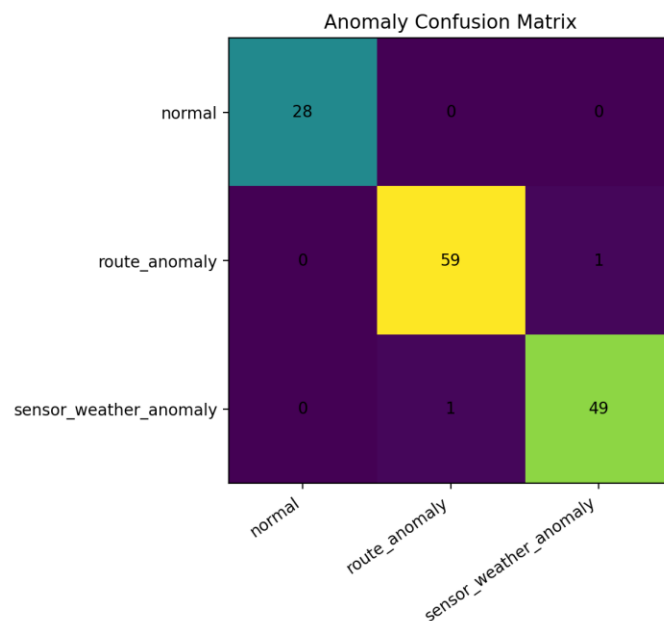


Figure 14. Anomaly confusion matrix for normal, route anomaly, and sensor/weather anomaly classes.

12. Dashboard Interface

The Streamlit dashboard integrates all system outputs in one interface. The sidebar allows the user to choose the routing algorithm, fleet budget, no-fly disruption row and column, and dense-area penalty option. The main dashboard displays the 3D city, CSP checks, routing algorithm comparison, live simulation, machine-learning results, drone telemetry, validation checks, 20-step simulation, and explanation tab.

Drone Telemetry Panel								
	Drone ID	Type	Hub	Battery	Payload	Max Payload	Speed	Status
0	D1	Light	1 1		100	1.2	2	1 idle
1	D2	Light	4 4		100	1.2	2	1 idle
2	D3	Light	5 6		100	1.2	2	1 idle
3	D4	Heavy	1 1		100	3.5	5	0.85 idle

In the simulation, route cost reduces battery. If anomaly is detected, drone status changes to returning_to_hub.

Figure 15. Drone telemetry panel showing drone type, hub, battery, payload, speed, and status.

13. Validation Checks and Output Artifacts

Automatic validation checks were added to verify that the system results are logically correct and defensible. The validation dashboard confirms CSP rule status, residential hub coverage, hub charging access, A* route existence, route start/end correctness, no-fly avoidance, and valid four-direction movement. All 10 checks passed in the observed run.

Automatic Validation Checks			
This tab verifies whether the AI results are logically correct and defensible.			
	Check	Status	Details
0	CSP R1	PASS	Industrial safety passed.
1	CSP R2	PASS	Residential hub coverage passed.
2	CSP R3	PASS	Hub charging access passed.
3	CSP R4	PASS	Hospital (4, 1) has medical pickup nearby at (5, 1).
4	Residential cells within 3 cells of hub	PASS	All residential cells covered
5	Every hub has charging pad within 2 cells	PASS	All hubs have nearby charging pads
6	A* route exists	PASS	Route cost=43.9, length=33
7	Route starts and ends at hub	PASS	Start=(1, 1), End=(1, 1), Expected Hub=(1, 1)
8	Route avoids no-fly cells	PASS	No no-fly violation
9	Route moves one cell at a time	PASS	All moves are valid 4-direction moves

Total Checks

10

Passed

10

Failed

0

All validation checks passed. Results are logically consistent.

Figure 16. Automatic validation checks showing 10 total checks, 10 passed, and 0 failed.

Output File	Purpose
csp_validation_results.csv	Stores pass/fail results and details for CSP rules R1-R4.
fleet_selection.csv	Stores selected light/heavy drone counts, cost, and total demand.
delivery_assignments.csv	Stores delivery-to-drone assignments and route status.
route_results.csv	Stores route coordinates by delivery and drone.
simulation_20_step_log.txt	Stores the 20-step simulation event log.
demand_forecasting_metrics.csv	Stores regression metrics such as MAE and RMSE.
demand_feature_importance.csv	Stores regression feature-importance values.
anomaly_classification_metrics.csv	Stores classifier comparison results.
anomaly_confusion_matrix.csv	Stores the anomaly confusion matrix.
aeronet_3d_animation.html	Standalone interactive 3D route animation viewable without Streamlit.

The HTML output is useful because it allows the 3D route animation to be opened directly in a web browser without running the Streamlit server. It also acts as a backup visual demo if the dashboard fails during presentation.

14. Conclusion

AeroNet Pro successfully implements the required AeroNet Lite drone delivery simulation. The project combines multiple AI techniques into an integrated and visual system. CSP validation ensures layout safety, fleet planning selects a feasible drone set under budget, A* and related search algorithms plan routes through the grid, no-fly disruption handling demonstrates real-time replanning, regression forecasts demand, and classification detects telemetry anomalies. The dashboard and exported output files provide clear evidence of implementation and results.

The final results show a complete working system: CSP PASS, 4 selected drones, route-planning success, 3 rerouted drones after disruption, demand forecasting with MAE 44.021 and RMSE 65.805, anomaly detection accuracy of 0.986, and a 20-step simulation summary with 4 completed deliveries, 0 delayed deliveries, and 0 failed deliveries.